

TEORETISK TENTAMEN i OPERATIVSYSTEM HI1025

Tentamen 240527

Inga hjälpmedel är tillåtna!

Svar lämnas på separata svarsapper. Tentamen består av 8 frågor om 6 poäng där lägsta poäng för E är ca 24 p.

Svara kortfattat och tydligt, rättningen drar poäng för fel eller saknad info - den ger inte poäng för rätt!

(Tips: 1 kiByte kräver 10 bitar, 1MiByte kräver 20 bitar)

1. Virtualisering av processorn I: Mekanismer

- Operativsystemet kan inte säkert tillåta att flera program körs till synes, eller verkligen parallellt, utan att det finns extra hårdvarustöd som inte behöver finnas i en processor som enbart ska köra bare metal program. Vad är det för principiella stöd? (3p)?
- När schemaläggaren återfått kontrollen ska den schemalägga någon process. Vart finns informationen som den baserar det beslutet på, och vad gör den om ingen användarprocess kan schemaläggas (2p)?
- När schemaläggaren beslutat vilken process som ska få köra närmast lämnar den över kontrollen till den processen via en speciell process som kallas ? och som ansvarar för att ? (bortse från sådant som är kopplat till virtuellt minne) (1p)?

2. Virtualisering av processorn II

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

- Ett operativsystem använder schemalägningspolicyn First Come First Serve (aka FIFO), hur skulle schemaläggaren schemalägga processer i tabellen ovan ? (rita bild!) och vad får de fyra processerna tillsammans för genomsnittlig turnaround time(1+1p)?
- Ett c-program som körs i en dator tilldelas vanligtvis tre minnesareor, hur benämns de tre minnesareorna (3*0.5p)?
- En av minnesareorna i deluppgift b) har en speciell koppling till funktionsanrop, där varje funktionsanrop leder till att en sådan ? placeras i minnesarean. Den egenskapen är av yttersta vikt för att kunna utveckla rekursiva algoritmer och benämns att funktionsanrop är ?. Förklara utförligt vad som händer om du kör programmet överst på nästa sida: (0.5+0.5+1.5p)

```
#include <stdio.h>

int mostFrequent(int throws[], int nb){
    int freqTbl[6];
    int i=0;
    int at=0;

    // Clear freqTable
    for (i=0; i<=6; i++)
        freqTbl[i]=0;

    // Mark throws
    for (i=0; i<=nb; i++)
        freqTbl[throws[i]-1]++;

    // Return most frequent number
    for (i=1; i<=6; i++)
        if (freqTbl[i]>freqTbl[at])
            at=i;
    return at+1;
}

int main(){
    int data[]={3,5,1,2,2,6};

    printf("Face %d most frequent\n", mostFrequent(data,6));
}
```

3. Virtualisering av minnet: Segment tekniken

- När en dator startar för första gången så "ägs" allt tillgängligt minne av os:et. Allt eftersom processer startas så måste OS:et dela ut minne och när processer avslutas återfår OS:et den avslutade processens minne. I ett OS som arbetar enligt **segment-tekniken** hanteras det ej utdelade minnet via en ? OS:et måste då också kunna hantera **2 relaterade processer** när den delar ut respektive återfår minnesblock – beskriv de två principiella processerna (1+1+1p)?
- Beskriv hur **rätt legal fysisk adress uppstår** i en dator när en process, som arbetar enligt segmenttekniken, utför en minnesaccess? (1p)?
- Du kör program till höger och får utskriften som följer efter programkoden. En stund senare, men en annan mix av program i gång på din dator kör du programmet igen: Vad skriver den ut den gången, motivering krävs, och går det att säga något om operativsystemets minneshantering (storlek) utifrån den utskriften? (1+1p)?

```
#include <stdio.h>

int main(void){
    int nb=0;

    printf("main() at %p\n", &main);
    printf("nb at %p\n", &nb);
}
```

```
main() at 0x10d79af40
nb at 0x7ff7b276856c
```

4. Virtualisering av minnet: Sidtekniken

- a) Förklara principiellt hur sidtekniken i praktiken fungerar, för de olika användningsfall som kan uppstå inkluderat inblandad hårdvara/mjukvara (2p)?
- b) Du arbetar med en processor där varje process tilldelas en virtuell adressrymd på 32 bitar. Sidstorleken är 4kByte och varje page table entry är 32 bitar. Hur stor hade en linjär page table blivit (i Byte)? (1p) Processorn arbetar i stället med en hierarkisk sidtabell i två nivåer där den första nivå motsvarar en adress 10 mest signifikanta bitar. Hur många sidtabeller blir det då maximalt totalt och hur stor är var och en av dessa sidtabeller? (0,5+0,5) Hur skulle de hierarkiska sidtabellerna se ut för ett typiskt C program, utförliga förklaringar krävs för full poäng? (2p)

5. Concurrency I

- a) Förklara noga vad som händer om efterföljande program körs, speciellt vad som skrivs ut (2p)?

b) Koden överst på nästa sida delar upp arbetet med att fylla en buffert med stjärnor '*' mellan 4 trådar! Skapa implementationen av funktionen doClear (som ska fylla en utpekad bit av strängen med stjärnor). (1.5p) Vad fyller rad 29 för funktion? (0.5p)

c) För de som siktar på A, en fråga som ni kan besvara om ni tänker stort utifrån det vi lärt oss i kursen: När en dator används sker hela tiden avbrott. En avbrottsrutin (ISR) påminner en del om en tråd (thread) men det finns (minst) två egenskaper kopplat till en avbrottsrutin som skiljer den från en tråd ur ett OS perspektiv, vilka två egenskaper är det? (2p)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  int counter=0;
6
7  void *plus1(void *vargp)
8  {
9      counter=counter+1;
10     return NULL;
11 }
12
13 void *plus2(void *vargp)
14 {
15     counter=counter+2;
16     return NULL;
17 }
18
19 int main(void)
20 {
21     pthread_t w1,w2;
22
23     pthread_create(&w1, NULL, plus1, NULL);
24     pthread_create(&w2, NULL, plus2, NULL);
25
26     printf("Counter is (%d)!\n", counter);
27     exit(0);
28 }
```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #define BSIZE 16
5  #define NTHREADS 4
6
7  typedef struct {
8      unsigned char *buffer;
9      int length;
10 } Buffer;
11
12 > void *doClear(void *vargp){ ...
13
14 void clearBuffer(unsigned char *p, int length){
15     pthread_t wt[NTHREADS];
16     Buffer pieces[NTHREADS];
17     for (int i=0; i<NTHREADS; i++) {
18         pieces[i].buffer=p+i*(length/4);
19         pieces[i].length=length/4;
20         pthread_create(&wt[i], NULL, doClear, (void *)&pieces[i]);
21     }
22     for (int i=0; i<NTHREADS; i++) {
23         pthread_join(wt[i], NULL);
24     }
25 }
26
27 int main(void) {
28     unsigned char buffer[BSIZE]={'\0'};
29     clearBuffer(buffer, BSIZE);
30     for (int i=0; i<BSIZE; i++)
31         printf("%c",buffer[i]);
32     printf("\nDone!\n");
33 }

```

OUTPUT TERMINAL PORTS DEBUG CONSOLE PROBLEMS

```

Anderss-MBP:vsws ac$ ./a.out
Working at 0x7ff7bb384554
Working at 0x7ff7bb384550
Working at 0x7ff7bb38455c
Working at 0x7ff7bb384558
*****
Done!

```

6. Concurrency II

a) Om du kör programmet till höger: Vad kommer det att skriva ut (bortsett från texten (PID:X)) och går det att säga något om ordningen på utskrifterna (ska motiveras)? (2p)

b) Det finns fyra kriterier som måste vara uppfyllda för att Deadlock ska kunna inträffa, vilka är det? (4*0,5p)

c) Koden som följer är från läroboken och visar först på en egendefinerad typ som kan användas för att

skapa en semaforliknande variabel. På rad 8 börjar en funktion för att initiera en variabel av den typen. Föreslå implementationen av wait funktionen för en dylik semafor, deklarationen är: void Zem_wait(Zem_t *s); (2p)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(void){
6      int child = fork();
7      int x=5;
8
9      if (child == 0) {
10         x += 5;
11         printf("x = %d (PID:%d)\n", x, (int)getpid());
12     } else {
13         child = fork();
14         x += 10;
15         if (child) {
16             x += 5;
17             printf("x = %d (PID:%d)\n", x, (int)getpid());
18         } else {
19             printf("x = %d (PID:%d)\n", x, (int)getpid());
20         }
21     }
22 }
```

```
1  typedef struct __Zem_t {
2      int value;
3      pthread_cond_t cond;
4      pthread_mutex_t lock;
5  } Zem_t;
6
7  // only one thread can call this
8  void Zem_init(Zem_t *s, int value) {
9      s->value = value;
10     Cond_init(&s->cond);
11     Mutex_init(&s->lock);
12 }
```

7. Persistens I

a) Du har tillgång till en väldigt stor hårddisk, flera TB. På hårddisken har du tänkt spara mätdata i form av mycket små filer (8 byte) med mättidpunkten som filnamn. Du har dividerat diskens storlek med 8 byte och kommit fram till att samtliga mätdatafiler ska rymmas med lite marginal. **Det finns minst 2 problem med det sättet att verifiera om informationen rymms,** vilka då (1+1p)?

b) En process utför en skrivoperation till en ansluten hårddisk som innebär att en befintlig fil ska utökas med 5000 Byte. Förklara principiellt vilken information som kommer att förändras på den anslutna hårddisken som arbetar med block om 4k Byte (2p)?

c) En typisk 320GB HD är Toshiba MK3254GSY. Under specifikationen för skrivhastighet till disk anges den som 54-128MB/s, hur kommer det sig att förmågan att skriva en hel fil till skivan kan ta olika lång tid (bortse från alla faktorer som är kopplade till datorn, utgå från att hårddisken har den data den behöver när den vill skriva till skivan) (2p)?

8. Persistens II

- a) Du har en folder med väldigt många filer. Vad heter kommandot för att lista alla filer i en folder ? (0.5p) Det finns också ett kommando för att räkna hur många rader det finns i en fil, vad heter det ? (0.5p). Hur kan de två kommandona kopplas samman för att ge svaret på hur många filer det finns i foldern ? (1p)
- b) Du håller på att utveckla ett system som kräver att en fil kan överföras felfritt och elektroniskt mellan 2 datorer på olika platser. **Vilket API** i din dator erbjuder lämpliga tjänster för att lösa det problemet, och **vilken information** behöver den första datorn ha för att kunna kontakta den andra (2p)?
- c) RPC var en tidig modell för att låta en process anropa en funktion som kunde finnas på en annan dator via något nätverk. RPC var tvungen att hantera flera svåra problem, beskriv kortfattat något (1p) Tekniken utvecklades vidare och anpassades till dagens Internet. Under Lab 4 sände ni ett Webservice meddelande, hur såg det principiellt ut? (1p)